A Synopsis

on

RAG BASED AI TEACHING ASSISTANCE

Submitted in partial fulfilment of the requirement for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE & ENGINEERING

Submitted by:

Keshav Goel	**2318977**
Anshika Mathur	**2318412**
Jatin Rana	**2318913**

Under the Guidance of

Dr. Susheela Dhaiya

Project Team ID: CSE27-107



Department of Computer Science and Engineering

Graphic Era Hill University

Dehradun, Uttarakhand

September - 2026

Graphic Era
HILL UNIVERSITY
Established by an Act of the State Legislature of Uttarakhand (Adhiniyam Sankhya 12 of 2011)

CERTIFICATE

This is to certify that the project synopsis entitled **"RAG Based AI Teaching Assistance"** is being submitted by **Keshav Goel (2318977), Anshika Mathur(2318412), Jatin Rana (2318913)**for the partial fulfilment of the requirement for the award of the degree of **B.Tech C.S.E.** in Department of Computer Science & Engineering, Graphic Era Hill University, Dehradun, and is a record of his/her own work carried out under my supervision. The matter embodied in the synopsis has not been submitted in any other university or institution for the award of any degree.

Dr. Susheela Dhaiya
Professor
Department of Computer Science and Engineering,
Graphic Era Hill University, Dehradun, India.

Graphic Era HILL UNIVERSITY
Established by an Act of the State Legislature of Uttarakhand (Adhiniyam Sankhya 12 of 2011)

CANDIDATE'S DECLARATION

We hereby certify that the work which is being presented in the project synopsis entitled **"RAG Based AI Teaching Assistance"** is being submitted for the partial fulfilment of the requirement for the award of the degree of **B.Tech C.S.E.** in Department of Computer Science & Engineering, Graphic Era Hill University, Dehradun, is our original work and has not been submitted to any other university or institution for the award of any degree or diploma.

Name	**Roll Number**	**Signature**
Keshav Goel	2318977	
Anshika Mathur	2318412	
Jatin Rana	2318913	

TABLE OF CONTENTS

Sr. No.	Description	Page No.
	Abstract	1
1	Introduction	2
2	Literature Review	4
3	Problem Statement & Objectives	6
4	Proposed Methodology	8
5	Hardware and Software Requirements	11
6	Expected Outcomes	12
7	Conclusion	13
	References	14

ABSTRACT

Lecture videos have become the most common way of delivering technical content, but they are also one of the hardest formats to search. A learner who wants to revise a single idea often has no choice but to scrub back and forth through an hour-long recording, and that friction is enough to make many students give up on revision altogether. General-purpose chatbots do not solve this problem either. They answer from whatever they picked up during training, so their explanations may be fluent but disconnected from the way a topic was actually taught in the videos a learner is studying.

This project proposes an AI Teaching Assistant built on Retrieval-Augmented Generation (RAG) that answers questions directly from a supplied library of lecture videos. The audio of every video is transcribed using a speech-to-text model that records the timing of each spoken segment. These timestamped transcripts are split into small, meaningful chunks, converted into embeddings, and stored in a vector database along with the video identifier and the start and end time of every chunk. When a learner asks a question, the system embeds the query, retrieves the closest chunks, and passes them to a large language model as grounding context so that the answer is written from the actual content of the videos rather than from general model knowledge.

The distinguishing feature of the proposed system is that every answer carries its source with it. Along with the explanation, the assistant reports the video and the exact timestamp at which the topic is discussed, so the learner can open the video at that second and watch the original explanation. The pipeline is content-agnostic and scales with whatever material is supplied, whether the library contains ten videos, twenty, fifty, or more. The expected outcome is a working web-based assistant that provides grounded answers with timestamp-level navigation, along with a reusable ingestion pipeline and a searchable transcript archive produced as a by-product.

CHAPTER 1

INTRODUCTION

1.1 Background

Over the last decade, recorded video has quietly become the default medium for teaching technical subjects. Universities record their lectures, online platforms publish complete courses, and independent instructors upload playlists that run into tens of hours. For a learner, this is an enormous amount of good material sitting in a format that computers, until recently, could not read.

The difficulty is one of access rather than availability. Text can be searched with a single keystroke; video cannot. A student who remembers that a particular idea was explained somewhere in a playlist has no reliable way to find it again. Video platforms offer titles, descriptions, and sometimes manually written chapter markers, but none of these operate at the level of the actual spoken content, and none of them understand a question phrased in a learner's own words.

Two developments have changed what is now possible. The first is automatic speech recognition that is accurate enough to transcribe ordinary lecture audio and, importantly, records when each phrase was spoken [2]. The second is Retrieval-Augmented Generation, an architecture in which a language model is supplied with relevant retrieved passages before it generates an answer [1]. Put together, these make it feasible to treat a video library as a searchable knowledge base and to answer questions from it in natural language.

1.2 Motivation

The motivation for this project comes from a very ordinary experience. A student watching a recorded lecture reaches a point they do not follow, and the person who could clarify it is not available. The explanation almost certainly exists somewhere in the recorded material, but locating it costs more time and patience than the doubt itself is worth.

Large language models look like an obvious answer to this, and in practice they are only a partial one. Asked about a technical concept, a general model will produce a competent textbook explanation, but it may use different notation, follow a different derivation, or skip the specific assumption the instructor emphasised. When the model has no relevant knowledge at all, it may still produce a confident answer, a failure mode widely

documented in the literature as hallucination [9]. For a learner who is still forming an understanding of the subject, a confident wrong answer is worse than no answer.

What is actually needed is an assistant that has, in effect, watched the videos. It should answer from what was taught, admit when a question is not covered by the material, and show the learner where its answer came from so that the claim can be checked. That combination of grounded answering and verifiable sourcing is what this project sets out to build.

1.3 Significance of the Proposed Work

The proposed system is significant for three reasons. First, it makes video content genuinely searchable at the level of ideas rather than filenames, which changes how a learner can revise. Second, by returning a timestamp with every answer, it treats the video as the authority and the assistant as a guide to it, keeping a human explanation in the loop instead of replacing it. Third, the design is content-agnostic: the same pipeline can be pointed at any collection of videos, which makes it useful well beyond the specific library it is first tested on.

1.4 Need for the Proposed System

- Existing video platforms support keyword search over titles and descriptions, not over the spoken content of a lecture.
- Manually prepared chapter markers are coarse, inconsistent, and rarely available for university recordings.
- General-purpose chatbots answer from training data and cannot be relied upon to reflect how a topic was taught in a particular set of videos.
- Learners need a way to move from a question to the exact moment of explanation in seconds, rather than scrubbing through a timeline.
- Instructors spend a disproportionate amount of time answering the same conceptual doubts that are already covered in their recorded material.

CHAPTER 2

LITERATURE REVIEW

This chapter reviews the work that the proposed system builds on, grouped into the three areas that matter for it: retrieval-augmented generation, speech recognition with timing information, and semantic retrieval over text.

2.1 Retrieval-Augmented Generation

The idea of giving a language model access to an external corpus at inference time was formalised by Lewis et al., who combined a dense retriever with a sequence-to-sequence generator and showed that the resulting model performed better on knowledge-intensive tasks than a purely parametric one [1]. Guu et al. approached the same problem from the pre-training side with REALM, learning the retriever jointly with the language model [8]. Both lines of work make the same underlying argument: a model does not need to memorise everything if it can look things up reliably.

A recent survey by Gao et al. traces how RAG has developed since then and identifies the parts of the pipeline that most affect output quality, particularly chunking strategy, retrieval accuracy, and prompt construction [7]. The survey also notes that retrieval grounding is one of the more practical defences against hallucination, which is consistent with the broader analysis of hallucination in generation systems by Ji et al. [9]. The present project follows this established architecture, with the addition that the retrieved units carry timing metadata.

2.2 Automatic Speech Recognition with Timestamps

Turning video into searchable text depends on transcription quality. Radford et al. introduced Whisper, a transformer-based speech recognition model trained on a large and varied multilingual corpus, which is robust to accents, background noise, and informal speech without task-specific fine-tuning [2]. For this project the significant property is not only accuracy but the fact that the model emits segment-level start and end times along with the text. Those timings are what make timestamp-level navigation possible; without them, a transcript would be searchable but would not point anywhere.

2.3 Dense Retrieval and Semantic Search

Keyword search fails when a learner phrases a question differently from the instructor, which is most of the time. Dense retrieval addresses this by representing both the query and the passages as vectors in a shared space, so that matching depends on meaning rather than shared words. Karpukhin et al. demonstrated with Dense Passage Retrieval that a dual-encoder trained on question-passage pairs substantially outperforms sparse retrieval on open-domain question answering [5]. Reimers and Gurevych made this practical at scale with Sentence-BERT, which produces sentence embeddings that can be compared with a simple cosine similarity [4]. Both build on the transformer architecture of Vaswani et al. [3] and the pre-training approach of Devlin et al. [10].

On the storage side, Johnson et al. showed with FAISS that similarity search over very large vector collections can be made fast enough for interactive use [6]. For a library of the size considered here, retrieval latency is not a bottleneck, but the same tooling allows the system to grow without redesign.

2.4 Comparison of Existing Approaches

Table 2.1 compares the approaches a learner might currently use against the system proposed in this project.

Approach	How it works	Limitation	Timestamp output
Platform keyword search	Matches text in titles, descriptions and tags	Does not search spoken content; fails on paraphrased queries	No
Manual chapter markers	Instructor labels sections of a video by hand	Labour-intensive, coarse-grained, often absent	Partial
General-purpose chatbot	Answers from parametric training knowledge	Not grounded in the given videos; can hallucinate	No
Transcript keyword search	Full-text search over an auto-generated transcript	Exact-match only; no synthesis or explanation	Yes
Proposed RAG assistant	Semantic retrieval over timestamped transcript chunks, followed by grounded generation	Depends on transcription quality for accuracy	Yes

Table 2.1 Comparison of Existing Methods with the Proposed System

2.5 Research Gap

The individual components are mature. Transcription with timings is a solved engineering problem, dense retrieval is well studied, and RAG is an established architecture. What is missing is the combination applied to lecture video with the timing metadata carried all the way through to the user. Most RAG systems in the literature return a document reference; a document reference is not useful for a fifty-minute recording. The gap this project addresses is therefore practical rather than theoretical: preserving segment-level timing through chunking, embedding, retrieval, and generation so that the final answer can point to a specific second of a specific video.

CHAPTER 3

PROBLEM STATEMENT & OBJECTIVES

3.1 Problem Statement

Learners studying from a library of lecture videos have no efficient way to find the specific portion of the material that answers a given question. Video platforms search metadata rather than spoken content, manually written chapter markers are coarse and frequently unavailable, and general-purpose language models answer from training data instead of the actual videos, which makes their explanations unreliable as a substitute for the taught material.

The problem, stated precisely, is to design and implement a system that accepts a natural-language question and a library of lecture videos of arbitrary size, and returns an answer that is generated strictly from the content of those videos, together with the identity of the source video and the exact timestamp at which the relevant explanation occurs, so that the learner can verify the answer against the original material.

3.2 Objectives

The objectives of the proposed work are as follows:

1. To build an end-to-end Retrieval-Augmented Generation pipeline that ingests a library of lecture videos and answers natural-language questions about their content.
2. To transcribe the videos using a speech-to-text model that produces segment-level timing information, and to preserve that timing through chunking, embedding, and retrieval.
3. To return, with every answer, the source video identifier and the start and end timestamp of the segment on which the answer is based, rendered as a clickable link wherever the hosting platform supports time-linked URLs.
4. To ground every generated response in the retrieved transcript content so that answers remain faithful to what was actually taught.
5. To detect questions that fall outside the indexed material and state this clearly to the learner instead of producing a speculative answer.
6. To keep the pipeline independent of library size, so that ten, twenty, fifty or more videos can be indexed without any change to the design.

7. To provide a simple web-based chat interface through which learners can ask questions and jump directly to the cited moment in the video.
8. To evaluate the system on retrieval accuracy, timestamp precision, answer faithfulness, and response latency using a prepared set of benchmark questions.

3.3 Scope of the Work

The scope of this project covers the ingestion and indexing of a supplied set of lecture videos, the retrieval and generation pipeline, timestamp-level source attribution, and a web interface for interaction. The system is intended to answer conceptual and explanatory questions about topics covered in the videos. It is not intended to grade assignments, generate new instructional content, or answer questions on material that has not been supplied to it, and it will explicitly say so when a question falls outside the indexed library.

CHAPTER 4

PROPOSED METHODOLOGY

The proposed system follows a two-phase Retrieval-Augmented Generation architecture. The first phase runs offline and converts the video library into a searchable index. The second phase runs at query time and produces a grounded answer with its source timestamp. Fig. 4.1 shows the overall workflow.

4.1 System Architecture

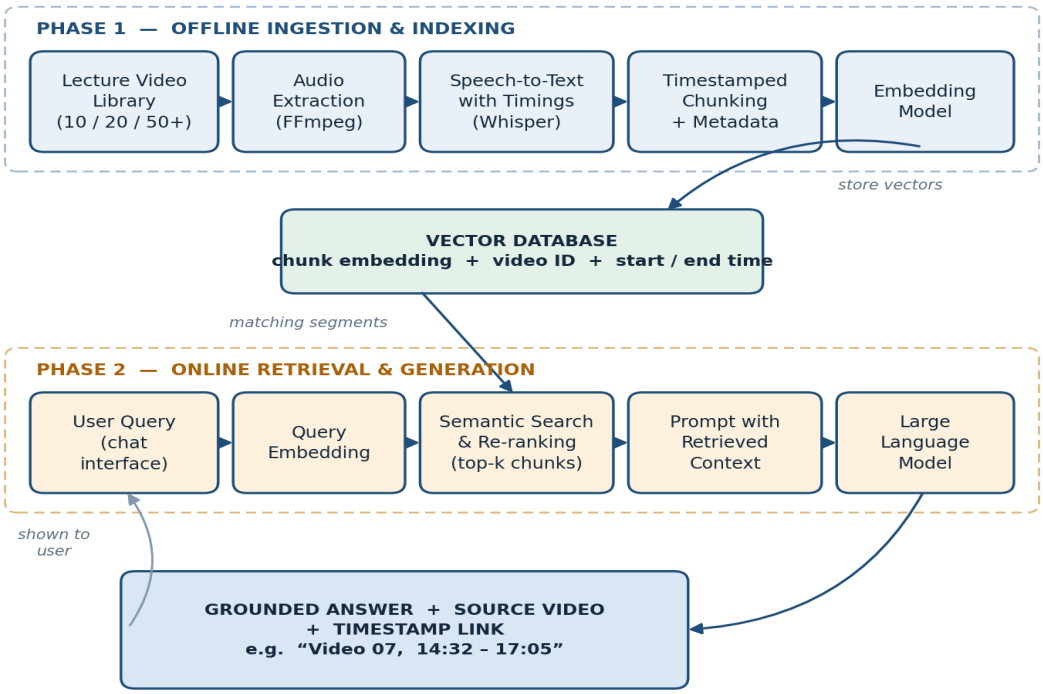


Fig. 4.1 Block Diagram of the Proposed System

4.2 Phase 1: Offline Ingestion and Indexing

4.2.1 Video Collection and Audio Extraction

The video files that form the library are collected and assigned stable identifiers. The audio track of each file is extracted using FFmpeg and converted to a uniform sampling rate and channel configuration, since transcription quality is sensitive to inconsistent input formats. The video identifier is recorded at this stage and travels with every artefact derived from that file.

4.2.2 Transcription with Timing Information

Each audio track is passed to a speech-to-text model that returns the transcript as a sequence of segments, where every segment carries its text along with a start time and an end time [2]. The output is stored as a structured transcript file per video. This step is the foundation of the entire system: the accuracy of the transcript determines the ceiling on retrieval quality, and the timing information is what makes the final navigation feature possible.

4.2.3 Timestamped Chunking

Raw transcript segments are usually too short to be meaningful on their own, so consecutive segments are grouped into chunks of a few hundred tokens with a small overlap between neighbouring chunks. The overlap prevents an explanation that straddles a boundary from being split in a way that makes neither half comprehensible. Each chunk stores the text together with its metadata: the video identifier, the start time of its first segment, and the end time of its last. Chunk size and overlap are treated as tunable parameters, since the survey literature identifies chunking strategy as one of the strongest influences on RAG output quality [7].

4.2.4 Embedding and Indexing

Every chunk is converted into a dense vector using a sentence embedding model [4]. The vector is stored in a vector database alongside its metadata, so that the timing information is retrieved automatically whenever the chunk is retrieved. Indexing is incremental: adding new videos to the library requires only that the new files be processed and appended, with no rebuilding of the existing index.

4.3 Phase 2: Online Retrieval and Generation

4.3.1 Query Embedding and Semantic Search

A learner's question is embedded using the same model used during indexing, which places the query in the same vector space as the chunks. A similarity search returns the top-k most relevant chunks [5], [6]. Because the comparison is semantic rather than lexical, a question phrased in the learner's own words will still match the instructor's phrasing of the same idea. An optional cross-encoder re-ranking step reorders the retrieved candidates to improve precision before they are passed on.

4.3.2 Prompt Construction and Grounded Generation

The retrieved chunks are inserted into a structured prompt together with the learner's question and a set of instructions that constrain the model's behaviour. The model is

directed to answer only from the supplied context, to state plainly when the context does not contain enough information to answer, and to avoid supplementing the answer with outside knowledge. This constraint is what keeps the assistant aligned with the taught material and is the principal defence against hallucination [1], [9].

4.3.3 Response Assembly and Timestamp Navigation

The generated answer is combined with the metadata of the chunks that produced it. The interface displays the explanation followed by the source video and the start and end time of the relevant segment, for example a reference to Video 07 between 14:32 and 17:05. Where the videos are hosted on a platform that supports time-linked URLs, the timestamp is rendered as a link that opens the video at that exact second; where the videos are served locally, an embedded player seeks to that position. If a topic is covered in more than one place, all relevant occurrences are listed in order of relevance.

4.4 Dataset and Preprocessing

The dataset for this project is the supplied lecture video library together with the transcripts generated from it. No externally labelled training data is required, since the system uses pre-trained models for transcription, embedding, and generation rather than training its own. Preprocessing consists of audio normalisation, removal of filler tokens and transcription artefacts, correction of recurring domain-specific terms through a small custom vocabulary, and the chunking described in Section 4.2.3. For evaluation, a benchmark set of representative questions is prepared with manually verified answers and reference timestamps.

4.5 Evaluation Approach and Performance Measures

The system will be evaluated on the following measures:

- **Transcription accuracy:** word error rate of the generated transcripts against a manually corrected sample.
- **Retrieval accuracy:** precision and recall at k, measuring how often the correct segment appears among the retrieved chunks.
- **Timestamp precision:** the difference in seconds between the cited start time and the point at which the topic actually begins in the video.
- **Answer faithfulness:** the proportion of generated statements that are supported by the retrieved context, assessed through manual review of the benchmark set.

- **Refusal correctness:** how reliably the system declines to answer questions that fall outside the indexed material.
- **Response latency:** end-to-end time from submission of a question to display of the answer.

CHAPTER 5

HARDWARE AND SOFTWARE REQUIREMENTS

5.1 Hardware Requirements

Transcription and embedding generation are the computationally demanding stages, and both run offline. Once the index has been built, answering a question is inexpensive. The requirements below therefore distinguish between the machine used to build the index and the machine used to serve it.

Component	Minimum	Recommended
Processor	Intel Core i5 (8th gen) or AMD Ryzen 5	Intel Core i7 / Ryzen 7 or higher
RAM	8 GB	16 GB or more
GPU	Not mandatory (CPU transcription is slower)	NVIDIA GPU with 6 GB VRAM or more, CUDA enabled
Storage	100 GB free (videos, audio, transcripts, index)	256 GB SSD or higher
Display	1366 x 768	1920 x 1080
Network	Stable broadband for hosted model APIs	Stable broadband, 10 Mbps or higher

Table 5.1 Hardware Requirements for the Proposed System

5.2 Software Requirements

Category	Tools and Technologies
Operating System	Windows 10/11, Ubuntu 20.04 LTS or later, or macOS
Programming Language	Python 3.10 or later
Audio Processing	FFmpeg for audio extraction and format conversion
Speech Recognition	OpenAI Whisper (or faster-whisper) with segment-level timestamps
RAG Orchestration	LangChain or LlamaIndex
Embedding Model	Sentence-Transformers (all-MiniLM-L6-v2 or similar) or a hosted embedding API
Language Model	Claude, GPT, or a locally hosted open-source LLM
Vector Database	FAISS or ChromaDB for local deployment; Pinecone for hosted deployment
Backend Framework	FastAPI or Flask
Frontend / Interface	Streamlit or React with an embedded video player
Supporting Libraries	NumPy, Pandas, PyTorch, Pydantic, python-dotenv
Version Control	Git and GitHub

Category	Tools and Technologies
Development Tools	Visual Studio Code, Jupyter Notebook, Postman

Table 5.2 Software Requirements for the Proposed System

CHAPTER 6

EXPECTED OUTCOMES

On completion, the project is expected to deliver the following:

1. **A working AI Teaching Assistant.** A web-based chat application that accepts natural-language questions and answers them from the indexed video library, with responses grounded in the retrieved transcript content.
2. **Timestamp-level navigation.** Every answer accompanied by its source video and the start and end time of the relevant segment, presented as a link or player seek so that the learner reaches the explanation in one click.
3. **A reusable ingestion pipeline.** An automated workflow that takes a folder of videos and produces a queryable index, usable on any new video library without modification to the code.
4. **A searchable transcript archive.** Complete timestamped transcripts of all processed videos, generated as a by-product of ingestion and independently useful for reading, revision, and accessibility.
5. **Out-of-scope detection.** Reliable identification of questions the indexed material does not cover, with a clear statement to that effect rather than a speculative answer.
6. **A performance evaluation report.** Measured results for retrieval accuracy, timestamp precision, answer faithfulness, and response latency on the prepared benchmark question set.
7. **Project documentation.** A record of the system architecture, setup instructions, configuration options, and observed limitations, sufficient for another team to reproduce and extend the work.

CHAPTER 7

CONCLUSION

This synopsis has proposed an AI Teaching Assistant that treats a library of lecture videos as a searchable knowledge base rather than an opaque collection of files. The system transcribes each video with segment-level timing, indexes the transcript as embedded chunks that retain their timing metadata, retrieves the most relevant segments for a learner's question, and generates an answer that is constrained to the retrieved material.

The contribution the project makes is the preservation of timing through the whole pipeline. A conventional retrieval-augmented system can tell a learner which document an answer came from; that is of limited help when the document is a fifty-minute recording. By carrying start and end times from transcription through to the final response, the proposed assistant can point to the precise moment an idea was explained, which turns the answer into a starting point for watching rather than a replacement for it.

The result is intended to be useful in two directions at once. Learners gain a way to resolve doubts at any hour and to navigate a large video library by meaning rather than by scrubbing a timeline. Instructors are relieved of repetitive conceptual questions that their recorded material already answers. Because the pipeline makes no assumptions about the subject of the videos and scales with whatever library it is given, the same system can be applied to any collection of recorded instruction, which makes the work a practical demonstration of Retrieval-Augmented Generation applied to video-based learning.

REFERENCES

[1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Kuttler, M. Lewis, W. Yih, T. Rocktaschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in Proc. 34th Int. Conf. Neural Information Processing Systems (NeurIPS), 2020, pp. 9459–9474.

[2] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, "Robust speech recognition via large-scale weak supervision," in Proc. 40th Int. Conf. Machine Learning (ICML), 2023.

[3] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in Proc. 31st Int. Conf. Neural Information Processing Systems (NeurIPS), 2017, pp. 5998–6008.

[4] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP-IJCNLP), 2019, pp. 3982–3992.

[5] V. Karpukhin, B. Oguz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W. Yih, "Dense passage retrieval for open-domain question answering," in Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP), 2020, pp. 6769–6781.

[6] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," IEEE Trans. Big Data, vol. 7, no. 3, pp. 535–547, 2021.

[7] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, and H. Wang, "Retrieval-augmented generation for large language models: A survey," arXiv preprint arXiv:2312.10997, 2023.

[8] K. Guu, K. Lee, Z. Tung, P. Pasupat, and M. Chang, "REALM: Retrieval-augmented language model pre-training," in Proc. 37th Int. Conf. Machine Learning (ICML), 2020, pp. 3929–3938.

[9] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung, "Survey of hallucination in natural language generation," ACM Computing Surveys, vol. 55, no. 12, pp. 1–38, 2023.

[10] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in Proc. NAACL-HLT, 2019, pp. 4171–4186.